

Introdução à Programação orientada a objetos

Marcos Monteiro Junior

Programando com Java

Sumário

0.1	Polimorfismo	3
0.2	Associação	3
0.2.1	Associação (Geral)	4
0.2.2	Agregação	5
0.2.3	Composição	6
Referências Bibliográficas		15

0.1 Polimorfismo

Em determinados momentos em uma hierarquia de classes, precisamos que um mesmo método de uma superclasse que foi herdado por uma subclasse se comporte de **forma diferente dependendo do objeto instanciado**. Isto surge devido à necessidade de flexibilidade que a hierarquia de classe deseja fornecer (Leite et al. (2016)).

A maior vantagem do uso do polimorfismo é que podemos utilizar objetos distintos e continuar executando o mesmo método, sendo que este método se moldará ao objeto corrente. Veja o exemplo abaixo, não necessariamente estará em nosso sistema final.

```
// Superclasse
class Pessoa {
    protected String nome;

    public void exibirPerfil(); // Metodo polimorfico
}

// Subclasse 1: Cliente
class Cliente extends Pessoa {
    @Override
    public void exibirPerfil() {
        System.out.println("Perfil do Cliente: " + this.nome);
    }
}

// Subclasse 2: Veterinário
class Veterinario extends Pessoa {
    @Override
    public void exibirPerfil() {
        System.out.println("Perfil do Veterinário: " + this.nome);
    }
}
```

O ato de redefinir um método herdado na subclasse é chamado de sobrescrita ou *override*. Diferentemente da sobrecarga (*overload*), em que métodos possuem o mesmo nome dentro da mesma classe, a sobrescrita ocorre com métodos de mesma assinatura situados em classes diferentes na hierarquia de herança. Assim, cada subclasse pode fornecer uma funcionalidade específica para aquele método, adaptando-o às necessidades de cada objeto.

0.2 Associação

Existem situações onde a herança não é a melhor escolha de relacionamento, lembrando que quando utilizamos herança dizemos que um objeto filho (subclasse) é um tipo do objeto específico da classe mãe (superclasse). E se precisarmos cadastrar o endereço dos veterinários e clientes? Conceitualmente, o endereço não é um tipo de cliente e ou

veterinário, muito menos um tipo de pessoa, logo o relacionamento entre eles é apenas de “ajuda”. Este tipo de relacionamento é chamado de associação.

Existem três tipos de associações que variam de acordo com dependência de um objeto com outro.

0.2.1 Associação (Geral)

É o relacionamento mais amplo, onde uma classe utiliza ou conhece a outra para realizar suas funções. Não implica, necessariamente, dependência de ciclo de vida.

Como exemplo, o veterinário atende um Pet. Ambos existem de forma independente, mas o veterinário possui uma associação temporária com o animal durante a consulta.

Um exemplo de código em Java para representar a associação entre um Veterinário e um Pet é apresentado abaixo:

```
class Veterinario {
    private String nome;
    //Demais atributos da classe Veterinário ...
    // Associação: Veterinário ‘tem um’ Pet
    private Pet pet;

    public Veterinario(String nome) {
        this.nome = nome;
    }

    public void atenderPet(Pet pet) {
        this.pet = pet;
        System.out.println(“Veterinário ” + this.nome + “ está
            atendendo o pet ” + pet.getNome());
    }
}
```

A associação geral é representada na UML por uma linha simples conectando as classes envolvidas, indicando que elas estão relacionadas de alguma forma, mas sem especificar a natureza exata dessa relação. A Figura 1 ilustra a associação entre as classes Veterinário e Pet.

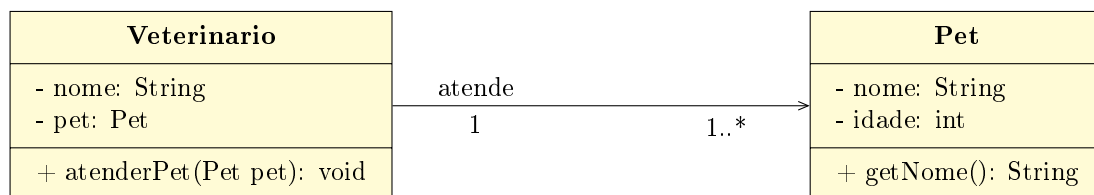


Figura 1: Exemplo de associação entre as classes Veterinário e Pet na UML.

0.2.2 Agregação

Representa um relacionamento “todo-parte”, mas é uma forma fraca de associação. As classes podem existir independentemente uma da outra. Se o “todo” for destruído, as “partes” continuam existindo.

Exemplo: Um Cliente e seus Pets. Se o cliente for removido do sistema, o objeto Pet pode persistir (por exemplo, vinculado a outro responsável ou ao histórico da clínica).

Vaja o código abaixo, representando a agregação entre Cliente e Pet:

```
public class Pet {  
    private String nome;  
    private String especie;  
    //Outros atributos da classe Pet...  
    public Pet(String nome,  
                String especie) {  
        this.nome = nome;  
        this.especie = especie;  
    }  
  
    public String getNome() {  
        return this.nome;  
    }  
}
```

```
public class Cliente {  
    private String nomeCliente;  
    // O pet existe independentemente  
    private Pet pet;  
    //construtor da classe Cliente solicitando um objeto Pet como  
    //parâmetro  
    public Cliente(String nome,  
                   Pet pet) {  
        this.nomeCliente = nome;  
        this.pet = pet;  
    }  
  
    public void exibirDados() {  
        System.out.println('‘Cliente: ’’ +  
                            this.nomeCliente +  
                            ‘‘ | Pet: ’’ + pet.getNome());  
    }  
}
```

Na UML, a agregação é representada por uma linha conectando as classes, com um losango vazio na extremidade da classe “todo”, indicando que a classe “parte” pode existir independentemente da classe “todo”. A Figura 2 ilustra a agregação entre as classes Cliente e Pet.

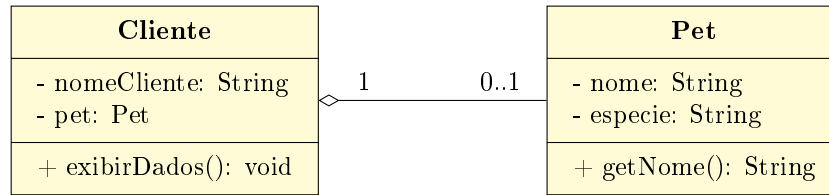


Figura 2: Representação UML da agregação entre as classes Cliente e Pet.

0.2.3 Composição

A composição é uma forma forte de associação, onde o “todo” é responsável pelo ciclo de vida das “partes”. Se o “todo” for destruído, as “partes” também são destruídas automaticamente. Nesse tipo de relacionamento, as “partes” não podem existir independentemente do “todo”.

Exemplo: O exemplo abaixo mostra a composição entre a classe Pessoa e a classe Endereco. O endereço é criado dentro da classe Pessoa, e se a instância de Pessoa for destruída, o endereço associado também será destruído. Além disso o exemplo mostra todos os tipos de relacionamento entre classes, como associação, agregação e composição.

```

import java.util.ArrayList;
import java.util.List;

// --- Composicao ---
class Endereco {
    private String rua;
    private String cidade;
    // 0 endereco nao existe fora da Pessoa (dependencia total)
    public Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }
}
  
```

```

class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // Composicao: Pessoa ‘tem um’ Endereco
    private Endereco endereco;

    public Pessoa(String nome, String cpf, String email, String
        rua, String cidade) {
        this.nome = nome;
        this.cpf=cpf;
        this.email=email;
        this.endereco = new Endereco(rua, cidade);
    }
}
  
```

```
class Consulta {
    private String data;
    // Agregacao: Consulta ‘tem varios’ Pets (o Pet existe
    // independentemente)
    private List<Pet> petsAtendidos = new ArrayList<>();

    public void adicionarPet(Pet p) {
        this.petsAtendidos.add(p);
    }
}
```

```
class Veterinario extends Pessoa {
    private String crmv;
    // Associacao de uso
    public void realizarConsulta(Consulta c) {
        System.out.println(‘‘Veterinario realizando consulta.’’);
    }
}
```

```
class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexo;
    //Demais atributos e métodos da classe Pet...
}
```

E no arquivo principal:

```
public class Main {
    public static void main(String[] args) {
        // Criando o veterinario com seu endereco (composicao)
        Veterinario vet = new Veterinario(‘‘Dr. Marcos’’, ‘‘Rua
        das Flores, 123’’, ‘‘Ponta Grossa’’);

        // Criando pets (agregados a consulta)
        Pet pet1 = new Pet(‘‘Rex’’, 3, 1);

        Consulta consulta1 = new Consulta(‘‘20/07/2026’’);
        consulta1.adicionarPet(pet1);

        vet.realizarConsulta(consulta1);
    }
}
```

Na UML a composição é representada por um losango preto, como mostrado na Figura 3, nela também se encontram todos os tipos de relacionamento entre classes, como associação, agregação e composição.

A Tabela 1 resume os tipos de relacionamento entre classes, suas características.

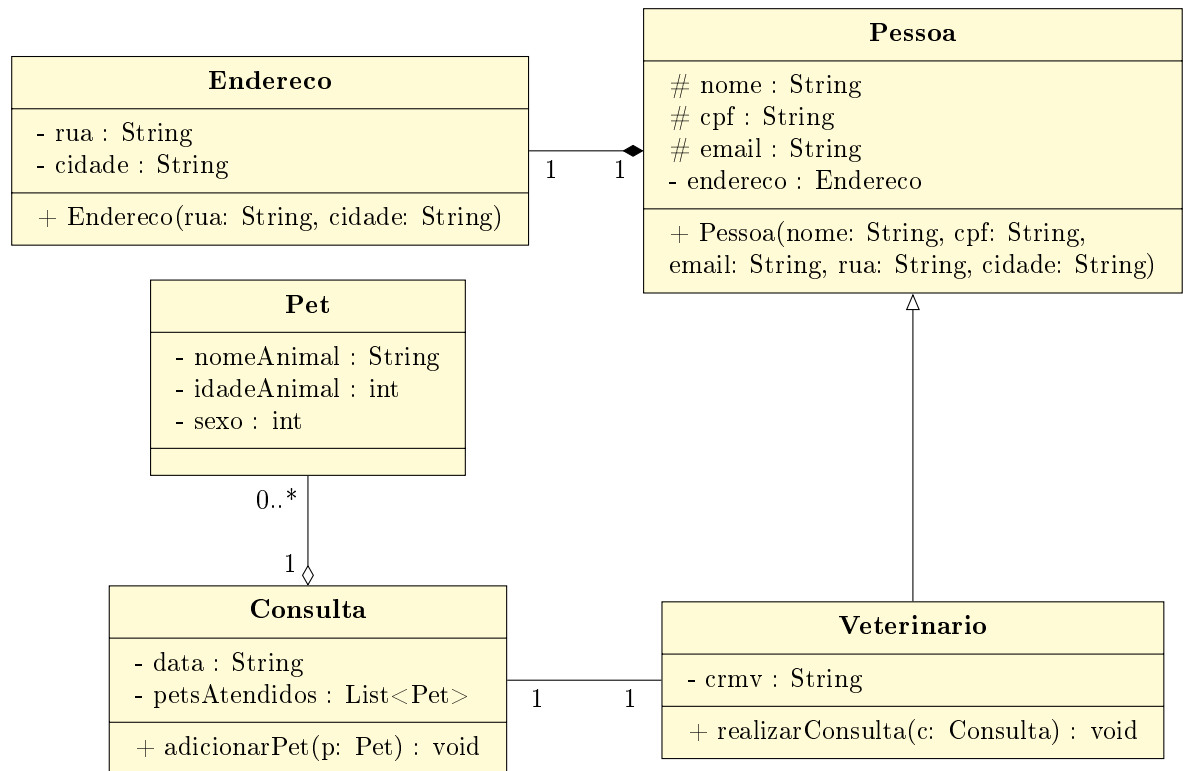


Figura 3: Diagrama UML representativo das classes Endereco, Pessoa, Veterinario, Consulta e Pet.

Relação	Conceito / Significado	Ciclo de Vida	Implementação em Java
Associação	Relacionamento fraco onde um objeto utiliza serviços ou dados de outro (“usa-um”).	Independentes: Os objetos existem e operam de forma totalmente autônoma.	Passagem do objeto como parâmetro de método (ex: ‘metodo(Objeto o)’).
Agregação	Relação do tipo “tem-um” (todo-parte), onde o todo contém partes, mas elas não dependem dele para existir.	Independentes: Se o objeto “todo” for destruído, a “parte” continua existindo no sistema.	Atributo de referência populado externamente, como listas ou coleções (ex: ‘List<Pet>’).
Composição	Relação de propriedade exclusiva e forte (todo-parte restrita). A parte pertence a um único todo.	Dependentes: Se o objeto “todo” for destruído, a “parte” é destruída junto obrigatoriamente.	Instanciação interna do objeto dentro do construtor do “todo” (ex: ‘this.parte = new Parte()’).

Tabela 1: Comparação entre Associação, Agregação e Composição.

Referências Bibliográficas

Leite, T. et al. (2016). *Orientação a Objetos: Aprenda seus conceitos e suas aplicabilidades de forma efetiva*. Editora Casa do Código.